

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent institution of MAHE, Manipal)

MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Management System Lab (MCA 4141)

TOY MANUFACTURING AND IMPORT DEPENDENCY ANALYSIS SYSTEM

SUBMITTED BY

STUDENT NAME	REG. NO.	SECTION	BATCH
Rakshitha	261090700029	A	A1
Ohil Raj S	261090700032	A	A1
Kishan Salian	261090700036	A	A1
Shreya R Shenoy	261090700039	A	A1

School of Computer Engineering
Manipal Institute of Technology, Manipal.

October 2026



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent institution of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the project titled “Toy Manufacturing and Import Dependency Analysis System” is a record of the bona fide work done by Student(s) submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications of Manipal Institute of Technology, Manipal, Karnataka, during the academic year 2026–2027.

Name and Signature of Examiners:

1._____

2. _____

TABLE OF CONTENTS

SECTION	PAGE
ABSTRACT	4
CHAPTER 1: INTRODUCTION	5
CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES	6
CHAPTER 3: FUNCTIONAL REQUIREMENTS – MODULES AND SOFTWARE USED	8
CHAPTER 4: ER DIAGRAM & RELATIONAL TABLES ALONG WITH SAMPLE DATA	10
CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS	15
CHAPTER 6: RESULTS & SNAPSHOTS	19
CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK	21
CHAPTER 8: REFERENCES	22

ABSTRACT

The Toy Manufacturing and Import Dependency Analysis System is a database-oriented mini project designed to organize information about toys, raw materials, suppliers, manufacturing batches, finished-toy stock, and sales transactions. The relational database represents the flow from raw-material sourcing to defining toy material requirements, recording production batches, maintaining stock, and recording sales.

Its distinctive feature is the comparison of local and imported sourcing options. The SUPPLIES relation records each supplier–material combination with source type, unit cost, country of origin, and availability status. These details allow imported sourcing to be compared with locally available alternatives. The system does not assume that local sourcing is always cheaper; comparisons are based on stored supplier prices.

The finalized conceptual model contains five main entities—SUPPLIER, RAW_MATERIAL, TOY, MANUFACTURING_BATCH, and SALE—and two associative relations, SUPPLIES and REQUIRES, to implement the many-to-many relationships.

CHAPTER 1: INTRODUCTION

1.1 Introduction

Toy manufacturing involves connected activities such as identifying required raw materials, selecting suppliers, recording production, monitoring finished-goods stock, and recording sales. When these details are maintained separately, it becomes difficult to obtain a consolidated view of sourcing, production, stock availability, and sales.

The proposed system provides a structured relational database for these activities. It links suppliers to raw materials, toys to their required materials, toys to manufacturing batches, and toys to sales transactions.

1.2 Need for the Proposed System

Centralized storage of supplier, raw material, toy, manufacturing batch, stock, and sales information.

Traceability of which suppliers provide particular raw materials and the sourcing cost and origin.

A record of the raw materials and quantities required for each toy.

Monitoring of finished-toy stock and identification of products at or below their reorder level.

A basis for reviewing production activity, sales value, and local-versus-imported sourcing costs.

1.3 Scope of the Project

The project is organized into four modules: Toy and Material Management; Supplier and Sourcing Management; Manufacturing and Inventory Management; and Sales and Business Analysis. It stores master details, supplier–material sourcing options, toy-wise material requirements, manufacturing batches, current stock, and sales transactions.

The project excludes live e-commerce APIs, online payment processing, customer account management, logistics tracking, customs processing, demand forecasting, and complete ERP functionality.

1.4 System Overview

The logical flow is: Supplier → SUPPLIES → Raw Material → REQUIRES → Toy → Manufacturing Batch. A toy is also linked directly to its sales records through SALE. SUPPLIES and REQUIRES are many-to-many relationships represented as junction tables during relational implementation.

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

A toy manufacturer needs connected information about products, raw materials, suppliers, production, stock, and sales. If supplier details and material prices are not linked to the materials required by each toy, identifying imported-material dependence and comparing local alternatives becomes difficult. Disconnected production, stock, and sales records also make it harder to review production quantities, stock availability, and sales performance.

This project addresses the problem by designing a relational database that connects supplier–material sourcing options, toy–material requirements, manufacturing batches, current stock, and sales transactions.

2.2 Objectives

To maintain toy and raw material records and define the quantity of each raw material required for a toy.

To maintain suppliers and associate them with materials; store source type, unit cost, origin, and availability.

To maintain production batch details with date, quantity, labour and overhead costs; maintain stock and reorder level in TOY.

To maintain sales details directly against a toy, including date, quantity, and unit selling price.

To compare local and imported sourcing options for the same material to support import-dependency and substitution analysis.

2.3 Existing System

A manual or spreadsheet-based approach may keep supplier details, material prices, toy requirements, production records, stock, and sales separately. This can cause repeated data, inconsistent updates, and difficulty tracing which suppliers provide a material or which toys depend on imported materials. Cost comparisons and stock or sales summaries may require manual consolidation.

2.4 Proposed System

The proposed system uses related tables to integrate these records. SUPPLIER and RAW_MATERIAL are connected through SUPPLIES, which stores sourcing-specific attributes. TOY and RAW_MATERIAL are connected through REQUIRES, which stores the quantity of each material required for a toy. A toy can have multiple manufacturing batches and multiple sales records. Current stock and reorder level are maintained in TOY.

2.5 Distinctive Feature: Import Dependency and Local Substitution Analysis

SUPPLIES stores source_type, unit_cost, country_of_origin, and availability_status for each supplier–material combination. A material can therefore have multiple sourcing options, including local and imported options. The system can compare the unit costs of available options for the same material and use the quantity in REQUIRES to estimate the cost difference for a toy. The comparison supports sourcing decisions but does not assume that a local option is always cheaper or available.

CHAPTER 3: FUNCTIONAL REQUIREMENTS - DIFFERENT MODULES AND SOFTWARE BEING USED

3.1 Modules of the System

Module	Objective / Function	Related Tables	Expected Output
1. Toy and Material Management	Maintain toy and raw material records and define the quantity of each raw material required for a toy.	TOY, RAW_MATERIAL, REQUIRES	Toy list, material list, toy-wise material requirement report.
2. Supplier and Sourcing Management	Maintain suppliers and associate them with materials; store source type, unit cost, origin, and availability.	SUPPLIER, RAW_MATERIAL, SUPPLIES	Supplier-wise supply details and local-versus-imported sourcing comparison.
3. Manufacturing and Inventory Management	Record production batches with date, quantity, labour and overhead costs; maintain stock and reorder level in TOY.	TOY, MANUFACTURING_BATCH	Manufacturing history, production summary, current stock, low-stock identification.
4. Sales and Business Analysis	Record sales directly against a toy, including date, quantity, and unit selling price.	TOY, SALE; analytical use of SUPPLIES and REQUIRES	Toy-wise sales summary, sales value, and indicative sourcing-cost comparison.

3.2 Functional Requirements

Insert, update, view, and maintain toy records, including current_stock and reorder_level.

Record supplier–material combinations with source type, unit cost, country of origin, and availability status.

Allow a supplier to provide multiple materials and a material to be supplied by multiple suppliers.

Allow a toy to require multiple materials and a material to be used in multiple toys.

Support queries to identify low-stock toys and compare local and imported sourcing options.

3.3 Non-Functional Requirements

Data integrity through primary keys, foreign keys, UNIQUE, NOT NULL, and suitable CHECK constraints.

A normalized relational structure that reduces unnecessary duplication and update anomalies.

Implementation using Oracle SQL and PL/SQL.

Readable and maintainable table and attribute names.

Efficient retrieval through joins, aggregate queries, and indexes where appropriate.

No dependence on external APIs or advanced infrastructure.

3.4 Hardware and Software Requirements

Category	Requirement
Operating System	Windows or another operating system supported by Oracle tools.
Database	Oracle Database 21c Express Edition (XE).
Development Tool	SQL*Plus, Oracle SQL Developer, or an equivalent Oracle database client.
Languages	SQL and PL/SQL.
Documentation	Microsoft Word or compatible software.
Hardware	A standard laptop or desktop capable of running Oracle XE and its client tools.

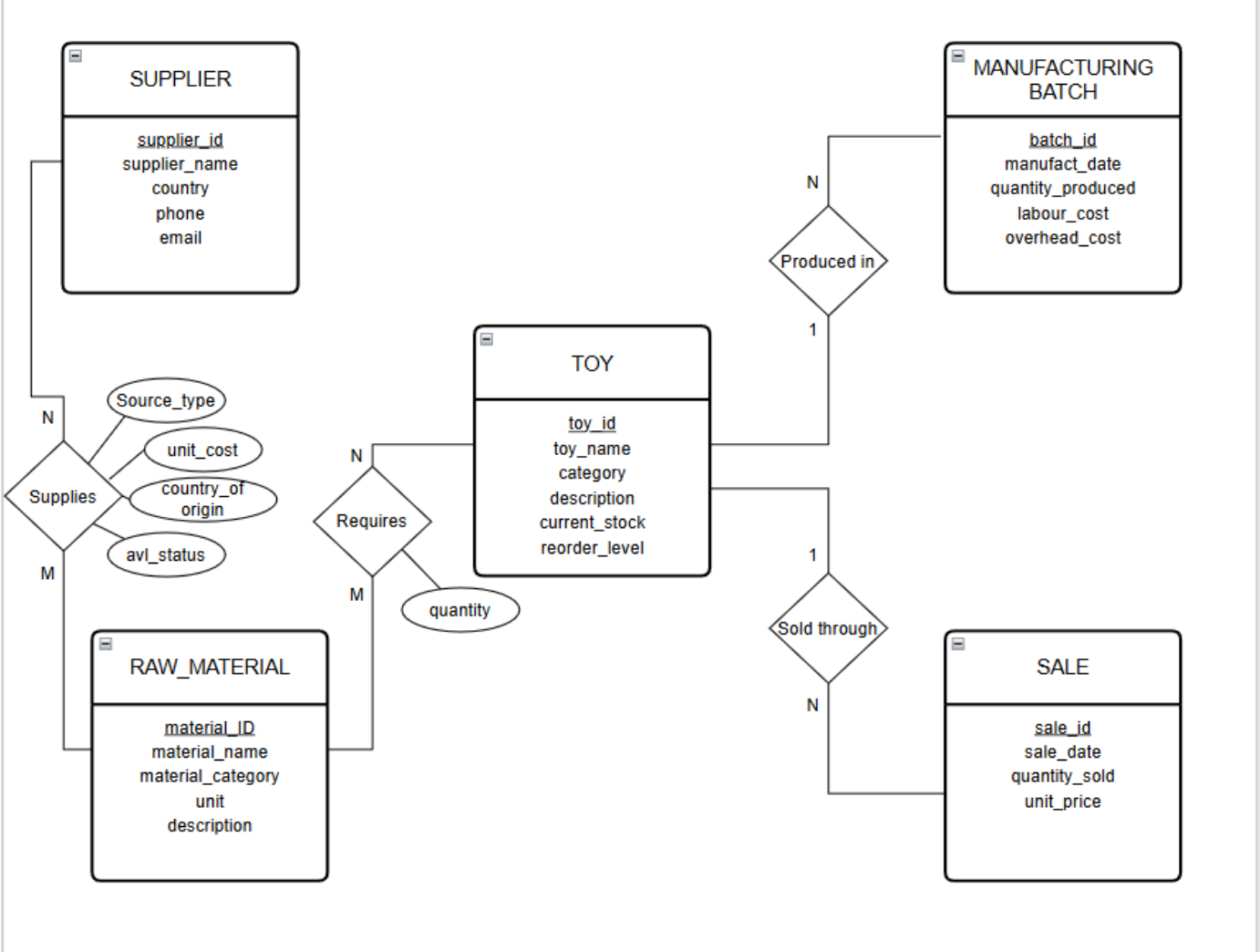
3.5 Overall System Workflow

Supplier → SUPPLIES → Raw Material → REQUIRES → Toy → Manufacturing Batch

Toy → SALE

CHAPTER 4: ER MODEL & RELATIONAL TABLES ALONG WITH SAMPLE DATA

4.1 ER Model



The conceptual ER model contains five main entities: SUPPLIER, RAW_MATERIAL, TOY, MANUFACTURING_BATCH and SALE.

4.2 Relational Schemas

SUPPLIER

SUPPLIER(supplier_id, supplier_name, country, phone, email)

PK: supplier_id;

RAW_MATERIAL

RAW_MATERIAL(material_id , material_name, material_category, unit, description)

PK: material_id;

TOY

TOY(toy_id, toy_name, category, description, current_stock, reorder_level)

PK: toy_id;

SUPPLIES

SUPPLIES(supplier_id , material_id, source_type, unit_cost, country_of_origin, availability_status)

Composite PK: (supplier_id, material_id).

REQUIRES

REQUIRES(toy_id , material_id, quantity_required)

Composite PK: (toy_id, material_id).

MANUFACTURING_BATCH

MANUFACTURING_BATCH(batch_id, toy_id **,** manufacturing_date, quantity_produced, labour_cost, overhead_cost)

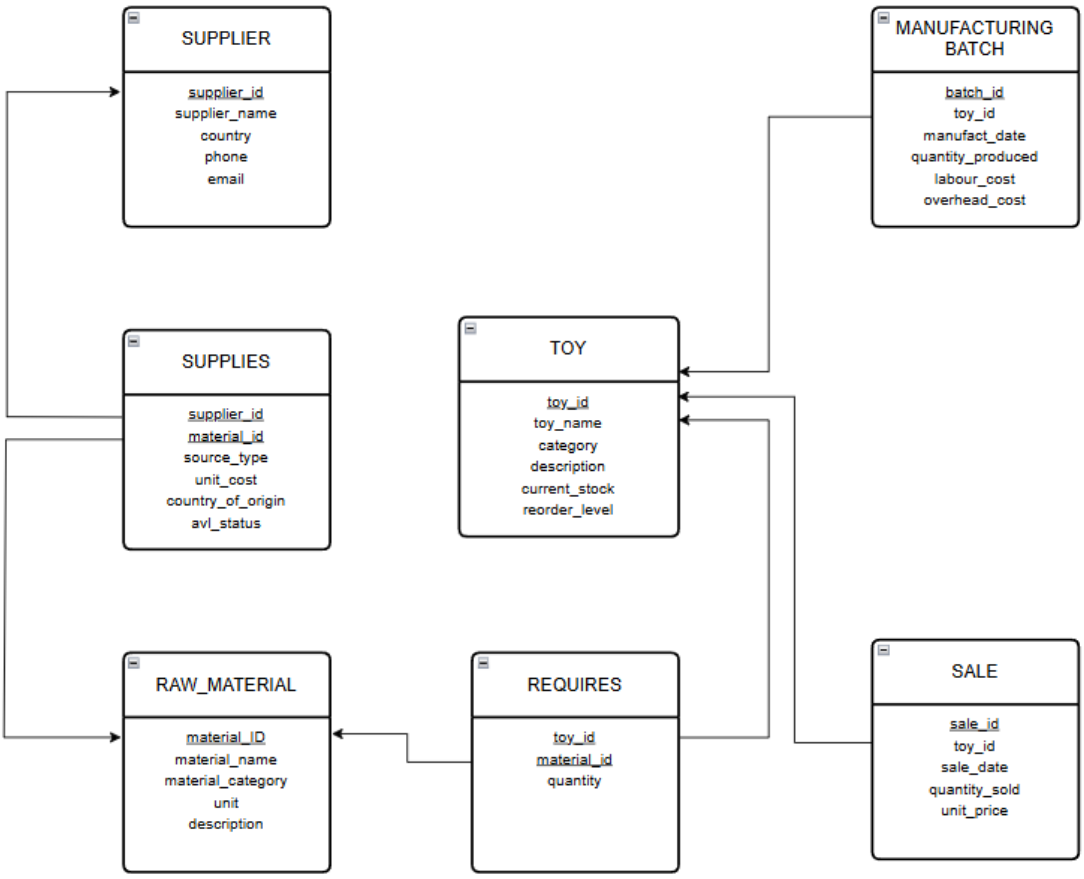
PK: batch_id;

SALE

SALE(sale_id , toy_id , sale_date, quantity_sold, unit_price)

PK: sale_id;

4.3 Schema Design



4.4 Normalization

Normalization is the process of organizing database tables to minimize data redundancy and prevent insertion, deletion, and update anomalies. The relational schema of the proposed system is designed to satisfy the First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

First Normal Form (1NF)

All tables satisfy First Normal Form because:

- Each table has a primary key to uniquely identify its records.
- Every attribute contains atomic (indivisible) values.
- There are no repeating groups or multivalued attributes.
- Each column stores values of a single attribute.

Therefore, all seven tables satisfy 1NF.

Second Normal Form (2NF)

All tables satisfy Second Normal Form because:

- Every table satisfies 1NF.
- In tables with single-column primary keys, all non-key attributes depend entirely on the primary key.
- In the SUPPLIES table, the attributes source_type, unit_cost, country_of_origin, and availability_status depend on the complete composite key (supplier_id, material_id).
- In the REQUIRES table, quantity_required depends on the complete composite key (toy_id, material_id).
- There are no partial functional dependencies of non-key attributes on a part of a composite primary key.

Therefore, all seven tables satisfy 2NF.

Normalization Summary

The relational schema of the Toy Manufacturing and Import Dependency Analysis System satisfies the following normalization forms:

- First Normal Form (1NF): Satisfied.
- Second Normal Form (2NF): Satisfied.

The resulting database structure reduces data redundancy, maintains data consistency, and minimizes insertion, deletion, and update anomalies.

4.5 Sample Data

Table: SUPPLIER

supplier_id	supplier_name	country	phone	email
1	Coastal Polymer Supplies	India	9876501001	sales@coastalpolymer.example
2	East Asia Components	China	861380010002	sales@eastasia.example
3	Western Metal Traders	India	9876501003	orders@westernmetal.example
4	SafeColor Industries	India	9876501004	support@safecolor.example
5	Bengal Textile Supply	Bangladesh	880170010005	sales@bengaltextile.example

Table: RAW_MATERIAL

material_id	material_name	material_category	unit	description
101	Plastic Granules	Polymer	kg	For molded toy parts
102	Steel Wire	Metal	kg	For axles and frames
103	Non-Toxic Paint	Finishing	litre	Child-safe finish
104	Cotton Fabric	Textile	metre	For soft toys
105	Electronic Module	Electronic	piece	For electronic toys

Table: TOY

toy_id	toy_name	category	description	current_stock	reorder_level
201	Building Blocks	Construction	Interlocking plastic blocks	90	20
202	Mini Racer Car	Vehicle	Small toy car with steel axle	68	15
203	Cuddle Doll	Doll	Fabric doll with painted details	55	12
204	Smart Robot	Electronic	Battery-powered toy robot	32	10
205	Plush Bear	Soft Toy	Stuffed teddy bear	61	15

Table: SUPPLIES

supplier_id	material_id	source_type	unit_cost	country_of_origin	availability_status
1	101	LOCAL	120.00	India	AVAILABLE
2	101	IMPORTED	105.00	China	AVAILABLE
3	102	LOCAL	80.00	India	AVAILABLE
4	103	LOCAL	150.00	India	AVAILABLE
5	104	IMPORTED	90.00	Bangladesh	LIMITED
1	104	LOCAL	98.00	India	AVAILABLE
2	105	IMPORTED	250.00	China	AVAILABLE

Table: REQUIRES

toy_id	material_id	quantity_required
201	101	0.200 kg
201	103	0.050 litre
202	101	0.150 kg
202	102	0.080 kg
203	104	0.600 metre
203	103	0.030 litre
204	105	1 piece
204	101	0.100 kg
205	104	0.500 metre
205	103	0.020 litre

Table: MANUFACTURING_BATCH

batch_id	toy_id	manufacturing_date	quantity_produced	labour_cost	overhead_cost
301	201	2026-08-01	100	4500.00	1800.00
302	202	2026-08-05	80	3200.00	1200.00
303	203	2026-08-12	60	3900.00	1500.00
304	204	2026-08-20	40	5200.00	2200.00
305	205	2026-08-25	70	2800.00	1100.00

Table: SALE

sale_id	toy_id	sale_date	quantity_sold	unit_price
401	201	2026-09-02	10	499.00
402	202	2026-09-04	12	349.00
403	203	2026-09-06	5	799.00
404	204	2026-09-10	8	1499.00
405	205	2026-09-12	9	599.00

CHAPTER 5: DQL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

5.1 Queries

Query 1: Compare local and imported sourcing options

```
SELECT r.material_name, s.supplier_name, p.source_type, p.country_of_origin, p.unit_cost, p.availability_status
FROM RAW_MATERIAL r JOIN SUPPLIES p ON r.material_id = p.material_id JOIN SUPPLIER s
ON p.supplier_id = s.supplier_id ORDER BY r.material_name, p.unit_cost;
```

Query 2: Calculate the estimated raw-material cost for each toy

```
SELECT t.toy_id, t.toy_name, COUNT(r.material_id) AS total_materials,  COUNT(s.material_id) AS
materials_with_source, SUM(r.quantity_required * s.min_cost) AS estimated_material_cost
FROM TOY t JOIN REQUIRES r ON t.toy_id = r.toy_id
LEFT JOIN ( SELECT material_id, MIN(unit_cost) AS min_cost FROM SUPPLIES
WHERE availability_status = 'AVAILABLE' GROUP BY material_id ) s
ON r.material_id = s.material_id GROUP BY t.toy_id, t.toy_name;
```

Query 3: Display manufacturing details and total production costs for each toy

```
SELECT t.toy_id, t.toy_name, COUNT(m.batch_id) AS total_batches,
    SUM(m.quantity_produced) AS total_quantity_produced,
    SUM(m.labour_cost) AS total_labour_cost,
    SUM(m.overhead_cost) AS total_overhead_cost,
    SUM(m.labour_cost + m.overhead_cost) AS total_production_cost
FROM TOY t JOIN MANUFACTURING_BATCH m ON t.toy_id = m.toy_id
GROUP BY t.toy_id, t.toy_name ORDER BY total_quantity_produced DESC;
```

Query 4: Calculate gross revenue generated by each toy

```
SELECT t.toy_id,
    t.toy_name,
    SUM(s.quantity_sold) AS total_quantity_sold,
    SUM(s.quantity_sold * s.unit_price) AS gross_revenue
FROM TOY t
JOIN SALE s ON t.toy_id = s.toy_id GROUP BY t.toy_id, t.toy_name
HAVING SUM(s.quantity_sold * s.unit_price) > 1000 ORDER BY gross_revenue DESC;
```

5.2 Function: Calculate Gross Sale Amount

Calculates the final payable amount for a specified sale by applying quantity-based discounts: 5% for purchases of 10–19 units and 10% for purchases of 20 or more units. No discount is applied for purchases below 10 units.

```
CREATE OR REPLACE FUNCTION sale_amount(p_sale_id IN NUMBER)
RETURN NUMBER
IS
   v_quantity SALE.quantity_sold%TYPE;
   v_price SALE.unit_price%TYPE;
   v_gross_amount NUMBER(12,2);
   v_discount NUMBER(5,2);
   v_final_amount NUMBER(12,2);
BEGIN
   -- Retrieve sale details
   SELECT quantity_sold, unit_price
   INTO v_quantity, v_price
   FROM SALE
   WHERE sale_id = p_sale_id;

   -- Calculate gross sale amount
   v_gross_amount := v_quantity * v_price;

   -- Determine discount based on quantity
   IF v_quantity >= 20 THEN
      v_discount := 10;
   ELSIF v_quantity >= 10 THEN
      v_discount := 5;
   ELSE
      v_discount := 0;
   END IF;

   -- Calculate final amount after discount
   v_final_amount := v_gross_amount -
                     (v_gross_amount * v_discount / 100);

   RETURN v_final_amount;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN NULL;
END;
/
```

5.3 Procedure: Record a Sale and Update Stock

Checks available stock, inserts the sale and decreases current_stock. The caller can COMMIT or ROLLBACK the transaction.

```
CREATE OR REPLACE PROCEDURE record_sale (
 p_sale_id IN NUMBER, p_toy_id IN NUMBER, p_sale_date IN DATE,
 p_quantity IN NUMBER, p_unit_price IN NUMBER
)
IS
 v_stock TOY.current_stock%TYPE;
BEGIN
 SELECT current_stock INTO v_stock
 FROM TOY WHERE toy_id = p_toy_id FOR UPDATE;

 IF p_quantity <= 0 OR p_quantity > v_stock THEN
  RAISE_APPLICATION_ERROR(-20001, 'Invalid quantity or insufficient stock');
 END IF;

 INSERT INTO SALE(sale_id, toy_id, sale_date, quantity_sold, unit_price)
 VALUES (p_sale_id, p_toy_id, p_sale_date, p_quantity, p_unit_price);

 UPDATE TOY SET current_stock = current_stock - p_quantity
 WHERE toy_id = p_toy_id;
END;
/
```

5.4 Trigger: Add Produced Units to Stock

Automatically increase the corresponding toy's stock by the quantity produced and display a warning if the updated stock is less than or equal to its reorder level.

```
CREATE OR REPLACE TRIGGER trg_batch_add_stock
AFTER INSERT ON MANUFACTURING_BATCH
FOR EACH ROW
DECLARE
  v_current_stock TOY.current_stock%TYPE;
  v_reorder_level TOY.reorder_level%TYPE;
BEGIN
  -- Update stock after manufacturing
  UPDATE TOY
  SET current_stock = current_stock + :NEW.quantity_produced
  WHERE toy_id = :NEW.toy_id;

  -- Retrieve updated stock and reorder level
  SELECT current_stock, reorder_level
  INTO v_current_stock, v_reorder_level
  FROM TOY
```

```
    WHERE toy_id = :NEW.toy_id;

    -- Check whether stock has reached the reorder level
    IF v_current_stock <= v_reorder_level THEN
        DBMS_OUTPUT.PUT_LINE(
            'WARNING: Toy ID ' || :NEW.toy_id ||
            ' has reached its reorder level. Current Stock: ' ||
            v_current_stock
        );
    ELSE
        DBMS_OUTPUT.PUT_LINE(
            'Stock updated successfully. Current Stock: ' ||
            v_current_stock
        );
    END IF;
END;
/
```

CHAPTER 6: RESULTS & SNAPSHOTS

The design supports supplier and raw-material sourcing management, toy material requirements, manufacturing batch records, current-stock tracking and toy-wise sales analysis.

6.1 Screenshots

Queries

```
SQL> SELECT r.material_name, s.supplier_name, p.source_type, p.country_of_origin, p.unit_cost, p.availability_status FROM RAW_MATERIAL r JOIN SUPPLIES p ON
r.material_id = p.material_id JOIN SUPPLIER s ON p.supplier_id = s.supplier_id ORDER BY r.material_name, p.unit_cost;

MATERIAL_NAME             SUPPLIER_NAME               SOURCE_TYPE  COUNTRY_OF_ORIGIN     UNIT_COST AVAILABILITY_STA
------------------------- --------------------------- ------------ -------------------- ---------- ----------------
Cardboard Sheets          Coastal Packaging           LOCAL        India                      8.50 LIMITED
Cotton Fabric             Shakti Textiles             LOCAL        India                     22.00 AVAILABLE
Electronic Modules        Bright Electronics          IMPORTED     Taiwan                    95.00 AVAILABLE
Non-toxic Paint           Bharat Polymers             LOCAL        India                     65.00 UNAVAILABLE
Plastic Granules          Bharat Polymers             LOCAL        India                     48.00 AVAILABLE
Plastic Granules          Global Plastics Ltd         IMPORTED     China                     53.50 AVAILABLE
Rubber Wheels             Coastal Packaging           LOCAL        India                     12.00 AVAILABLE
Steel Springs             Euro Metal Supplies         IMPORTED     Germany                   14.75 AVAILABLE
Wooden Blocks             Pacific Components          IMPORTED     Vietnam                   29.50 LIMITED
Wooden Blocks             Karnataka Wood Works        LOCAL        India                     32.00 AVAILABLE

10 rows selected.

SQL> SELECT t.toy_id, t.toy_name, COUNT(r.material_id) AS total_materials, COUNT(s.material_id) AS materials_with_source, SUM(r.quantity_required * s.min_co
st) AS estimated_material_cost FROM TOY t JOIN REQUIRES r ON t.toy_id = r.toy_id LEFT JOIN (SELECT material_id, MIN(unit_cost) AS min_cost FROM SUPPLIES WHE
RE availability_status = 'AVAILABLE' GROUP BY material_id
  2  ) s ON r.material_id = s.material_id GROUP BY t.toy_id, t.toy_name;

 TOY_ID TOY_NAME                 TOTAL_MATERIALS MATERIALS_WITH_SOURCE ESTIMATED_MATERIAL_COST
------- ------------------------ --------------- -------------------- ------------------------
    201 Building Blocks Set                    2                    1                       12
    205 Mini Toy Car                           2                    2                     52.8
    202 Wooden Puzzle                          2                    1                     12.8
    208 Wooden Train Set                       1                    1                     25.6
    203 Soft Teddy Bear                        1                    1                     26.4
    204 Wind-up Robot                          1                    1                     29.5
    206 Musical Toy Phone                      1                    1                       95

7 rows selected.

SQL>
```

Procedure

```
SQL> SELECT toy_id, toy_name, current_stock FROM TOY WHERE toy_id = 201;

 TOY_ID TOY_NAME                 CURRENT_STOCK
------- ------------------------ -------------
    201 Building Blocks Set                160

1 row selected.

SQL> @ D:\MCA\DBMS\Mini_proj\COMMANDS\PR_1.txt;

Procedure created.

SQL> EXEC record_sale(4013, 201, SYSDATE, 5, 250);

PL/SQL procedure successfully completed.

SQL> SELECT * FROM SALE WHERE sale_id = 4013;

SALE_ID  TOY_ID SALE_DATE     QUANTITY_SOLD UNIT_PRICE
------- ------- ------------- ------------- ----------
   4013     201 01-NOV-26                 5     250.00

1 row selected.

SQL> SELECT toy_id, toy_name, current_stock FROM TOY WHERE toy_id = 201;

 TOY_ID TOY_NAME                 CURRENT_STOCK
------- ------------------------ -------------
    201 Building Blocks Set                155

1 row selected.

SQL>
```

Trigger

```
SQL> INSERT INTO MANUFACTURING_BATCH VALUES (3019, 203, SYSDATE, 1, 500, 150);
WARNING: Toy ID 203 has reached its reorder level. Current Stock: 24

1 row created.

SQL> SELECT toy_id, toy_name, current_stock FROM TOY WHERE toy_id = 203;

 TOY_ID TOY_NAME                  CURRENT_STOCK
------- ------------------------- -------------
    203 Soft Teddy Bear                      24

1 row selected.

SQL> select * from toy;

 TOY_ID TOY_NAME                  CATEGORY        DESCRIPTION                                    CURRENT_STOCK REORDER_LEVEL
------- ------------------------- --------------- ---------------------------------------------- ------------- -------------
    209 Flexible Animal Figures   Educational     Soft flexible animal-shaped figures made using r             0            15
    210 Bamboo Building Kit       Educational     Lightweight construction kit made from processed            20            12
    211 Light-up Toy Car          Electronic      Toy car with LED light and sound features                   63            15
    212 Eco-friendly Toy House    Creative        Wooden toy house supplied in recycled cardboard             50            10
    201 Building Blocks Set       Educational     Colourful interlocking plastic construction bloc           245            30
    202 Wooden Puzzle             Educational     Wooden shape and pattern puzzle                             90            20
    203 Soft Teddy Bear           Soft Toy        Cotton-filled soft teddy bear                               24            25
    204 Wind-up Robot             Mechanical      Mechanical robot toy with spring mechanism                  75            20
    205 Mini Toy Car              Vehicle         Small toy car with rubber wheels                           150            40
    206 Musical Toy Phone         Electronic      Toy phone with basic sound and light features               60            15
    207 Colouring Toy Kit         Creative        Children painting and colouring activity kit                45            15
    208 Wooden Train Set          Vehicle         Wooden train set with multiple carriages                    80            25

12 rows selected.
```

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

The Toy Manufacturing and Import Dependency Analysis System organizes supplier, raw-material, toy, manufacturing and sales information in a compact relational design. Supplies records sourcing type, origin, cost and availability, while Requires records the quantity of each material needed per toy. The database supports comparisons of local and imported sourcing options, basic stock management, manufacturing records and gross sales analysis. Oracle constraints and PL/SQL routines help maintain data integrity and automate selected operations.

7.2 Limitations

- The model does not record actual material consumption for each manufacturing batch.
- Sales are linked directly to toys, so individual sales cannot be traced to a specific production batch.
- Inventory is stored as a current-stock value; stock movement history and multiple warehouses are not represented.
- Sales channels, discounts and platform fees are not separately modeled.
- The sourcing comparison does not automatically determine whether different materials are technically interchangeable.

7.3 Future Work

- Add batch-level material usage to record actual quantities and costs consumed during production.
- Add sales channels, discounts and platform fees for more detailed net-revenue analysis.
- Maintain inventory transaction history and automated low-stock alerts.
- Generate reports comparing estimated production costs under local and imported sourcing scenarios.
- Add supplier lead times and validated material-substitution rules.

CHAPTER 8: REFERENCES

The following standard references cover database design and the Oracle technologies used in the project.

1. Oracle. Oracle Database SQL Language Reference, 21c. Oracle Documentation.
2. Oracle. Oracle Database PL/SQL Language Reference, 21c. Oracle Documentation.
3. Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems. Pearson.
4. Silberschatz, A., Korth, H. F., and Sudarshan, S. Database System Concepts. McGraw Hill.